GABEO AI

ML Engineer
Take-Home Assignment

AI-Powered Claim Denial Analysis for Healthcare RCM

Deadline: **3 Calendar Days**
April 2026

CONFIDENTIAL
This document is proprietary to Gabeo AI. Do not distribute.

1. The Problem

Healthcare providers in the US lose billions of dollars annually to insurance claim denials. When a provider submits a claim to an insurance payer, the payer may deny the claim for a variety of reasons: missing information, timely filing violations, medical necessity disputes, duplicate submissions, incorrect coding, and more.

Today, billing teams manually review each denial, research the reason, compare it against historical patterns and payer policies, and decide whether to appeal. This process is slow, expensive, and inconsistent.

Your job is to build an AI system that automates this analysis.

What We Give You

EDI 835 data schema (remittance advice — what the payer paid and why)

EDI 837 data schema (original claim submission — what was billed and for what)

CARC/RARC code references (standardized denial reason codes)

Sample denied claims with realistic scenarios

A clear set of problems to solve

What We Do NOT Give You

We do not prescribe any specific architecture, framework, or agent design.

How you structure your AI system, what models you use, how agents interact (or whether you use agents at all) is entirely up to you.

We want to see YOUR thinking. A working demo is a strong plus.

2. The Data

Our system works with two primary EDI (Electronic Data Interchange) data sources. These are industry-standard healthcare transaction formats. Below are the key fields from each — the full schemas are provided as attached CSVs (835_Attributes.csv, 837_Attributes.csv).

2.1 EDI 835 — Remittance Advice (Payer's Response)

This is what the insurance company sends back after processing a claim. It tells you what was paid, what was denied, and why. The 835 has three levels of data:

Payment-Level Fields (cp_ prefix)

Top-level payment information from the payer to the provider.

Field	Description
cp_PayerName	Name of the insurance payer (e.g., Blue Cross, Aetna, Medicare)
cp_PaymentAmount	Total payment amount on the remittance
cp_PaymentMethod	How payment was sent (check, EFT, etc.)
cp_PayerID	Unique payer identifier
cp_PayeeName	Provider/facility receiving the payment
cp_TotalClaimCount	Number of claims in this remittance
cp_TotalDeniedChargeAmount	Total dollar amount denied across all claims
cp_TotalClaimChargeAmount	Total billed amount across all claims
cp_EffectiveDate	Date the payment was processed

Claim-Level Fields (pc_ prefix)

Per-claim adjudication details within a remittance.

Field	Description
pc_ClaimID	Unique claim identifier — joins to 837 data
pc_ClaimStatus	Claim status code (1=Processed as Primary, 2=Processed as Secondary, 3=Processed as Tertiary, 4=Denied, 19=Processed as Primary, Forwarded, 22=Reversal)
pc_ClaimAmount	Total billed amount for this claim
pc_ClaimPaid	Amount actually paid by the payer
pc_PatientResponsibility	Amount the patient owes
pc_InsuranceType	Type of insurance (Medicare, Medicaid, Commercial, etc.)
pc_StatementBegin / End	Date range for services on this claim
pc_ReceivedDate	When the payer received the claim
pc_PriorAuthNum	Prior authorization number, if applicable
pc_PatientLast / First	Patient name
pc_RenderingID	Rendering provider NPI

Line-Level Fields (pcl_ prefix)

Individual service line details within a claim.

Field	Description
pcl_ProcedureCode	CPT/HCPCS procedure code for the service
pcl_ProcedureModifier1-4	Procedure modifiers (affect reimbursement)
pcl_ChargedAmount	Amount billed for this service line

Field	Description
pcl_PaidAmount	Amount paid for this line
pcl_AllowedAmount	Payer-approved amount
pcl_ServiceDate	Date of service
pcl_RemarkCodes	RARC remark codes with additional context

Adjustment Fields (pcla_ prefix)

This is the most critical section for denial analysis. Each line can have multiple adjustments.

Field	Description
pcla_AdjustmentGroup	CO (Contractual Obligation), PR (Patient Responsibility), OA (Other Adjustment), PI (Payer Initiated), CR (Correction/Reversal)
pcla_AdjustmentReason	CARC code — the standardized reason for the adjustment (e.g., 16, 29, 50, 97)
pcla_AdjustmentAmount	Dollar amount of the adjustment
pcla_AdjustmentQty	Quantity of units adjusted

2.2 EDI 837 — Claim Submission (What Was Originally Billed)

This is the original claim submitted by the healthcare provider. It contains patient demographics, diagnosis codes, procedure details, provider info, and more. The 837 has two levels:

Claim-Level Fields (ec_ prefix)

The claim header with patient, provider, payer, and diagnosis information.

Field	Description
ec_ClaimNo	Claim number — joins to 835 pc_ClaimID
ec_Amount	Total billed amount
ec_PlaceOfService	Where service was rendered (office, hospital, etc.)
ec_PayerName / ec_PayerID	Insurance payer information
ec_InsuranceType	Medicare, Commercial, Medicaid, etc.
ec_PrincipalDiagnosis	Primary ICD-10 diagnosis code
ec_Diag2 through ec_Diag25	Additional diagnosis codes
ec_BillProvNPI	Billing provider National Provider Identifier
ec_RendProvNPI	Rendering provider NPI
ec_ServiceDateFrom / To	Date range of services
ec_PriorAuthorization	Prior auth number, if obtained
ec_TypeOfBill	Facility bill type code

Field	Description
ec_ClaimFrequency	1=Original, 7=Replacement, 8=Void
ec_DelayReasonCode	Why the claim was submitted late, if applicable
ec_PatientRelationship	Patient relationship to subscriber
ec_SubscriberID	Insurance subscriber/member ID

Service Line Fields (cd_ prefix)

Individual procedure lines on the claim.

Field	Description
cd_ProcedureCode	CPT/HCPCS code for the service performed
cd_Modifier1 through Modifier4	Procedure modifiers
cd_Amount	Billed amount for this line
cd_Quantity	Number of units
cd_DiagPointer1-4	Pointers to diagnosis codes on the claim header
cd_PlaceOfService	Place of service for this line
cd_ServiceDateFrom / To	Service date range
cd_RevenueCode	Revenue code (facility claims)
cd_PriorAuthNo	Line-level prior auth

2.3 How 835 and 837 Connect

The claim ID is the key link: 835's pc_ClaimID matches 837's ec_ClaimNo. For a denied claim, you need BOTH datasets to understand what happened:

- 835 tells you: what was denied, how much, and the reason codes (CARC/RARC)
- 837 tells you: what was originally billed, the diagnoses, the procedures, the provider
- Together they give you the full picture needed to determine if recovery is possible

2.4 CARC/RARC Codes — Denial Reason Codes

These are standardized codes used across the US healthcare industry. The CARC (Claim Adjustment Reason Code) tells you WHY a claim was adjusted. The group code tells you WHO is financially responsible.

Common CARC Codes You Will Encounter

Code	Category	Meaning
4	Coding	The procedure code is inconsistent with the modifier used, or a required modifier is missing

Code	Category	Meaning
16	Information	Claim/service lacks information needed for adjudication
18	Duplicate	Exact duplicate claim/service
29	Timely Filing	The time limit for filing has expired
50	Medical Necessity	These are non-covered services because this is not deemed a medical necessity
96	Non-Covered	Non-covered charge(s)
97	Duplicate	The benefit for this service is included in the payment for another service that was already adjudicated
197	Precertification	Precertification/authorization/notification absent
252	Information	An attachment/other documentation is required to adjudicate this claim

Group Codes

Group	Name	What It Means
CO	Contractual Obligation	Provider agreed to this adjustment per contract — cannot bill patient
PR	Patient Responsibility	Patient owes this amount (deductible, coinsurance, copay)
OA	Other Adjustment	No party is assigned financial responsibility
PI	Payer Initiated	Payer-initiated reduction not tied to contract

---

3. The Problems to Solve

Using the data described above, build an AI-powered system that can perform the following. You are free to tackle these in any order, but they are listed by importance.

Problem 1: Claim Denial Root Cause Analysis (Must Have)

Given a denied claim (835 data joined with its corresponding 837 data), your system should automatically determine:

- What is the root cause of the denial? (Go beyond just reading the CARC code — analyze the claim data to understand WHY this specific claim was denied)
- Is this denial valid or is it potentially recoverable? And why?
- What specific evidence from the claim data supports your analysis?

Example Scenario

A claim comes in denied with CARC 29 (timely filing). Your system should look at the 837's ec_ServiceDateFrom, the 835's pc_ReceivedDate, and determine: Was it actually filed late? Does the 837 have a ec_DelayReasonCode? Is this a Medicare claim (365-day filing limit) or a Commercial claim (often 90–180 days)? Could this be a secondary claim where timely filing starts from the primary payer's EOB date?

Expected Output

A structured JSON output per claim containing at minimum: denial root cause (human-readable), CARC/RARC interpretation, recoverability verdict (recoverable, not recoverable, needs review), confidence score, and supporting evidence pulled from the actual claim fields.

Problem 2: Historical Pattern Matching (Must Have)

Given a set of claims (both paid and denied), your system should be able to:

- Find historically paid claims that are similar to a newly denied claim (same payer, similar procedures, similar diagnosis codes, same provider)
- Use those patterns to strengthen or weaken the case for recoverability
- Identify systemic denial patterns (e.g., "This payer denies CPT 99213 with modifier 25 at a 40% rate")

What We Are Looking For

How you define "similarity" between claims, how you handle the high-dimensional and sparse nature of claim data, and how you use historical context to make better predictions. Think about which fields matter most for matching and why.

Problem 3: Denial Clustering & Batch Intelligence (Should Have)

Billing teams do not work one claim at a time. They have thousands of denials to process. Your system should:

- Group denied claims into meaningful clusters (by denial reason, payer, procedure type, or a combination)
- Identify the highest-value clusters (where the most money can be recovered)
- Produce a summary that tells a billing team: "You have 45 claims from Aetna denied for missing prior auth, totaling $128,000. Based on historical data, 70% of similar claims were successfully appealed."

4. Sample Denied Claims

Use the following sample claims to build and test your system. You should also generate additional synthetic claims to create a larger test set (we recommend 20–50 claims with a mix of paid and denied).

Claim A — Timely Filing Denial

```
// 835 Data (Remittance)
{  "pc_ClaimID": "CLM-2026-00142",
   "pc_ClaimStatus": "4",
   "pc_ClaimAmount": 4500.00,
   "pc_ClaimPaid": 0.00,
   "pc_InsuranceType": "Commercial",
   "pc_ReceivedDate": "2026-03-20",
   "pc_StatementBegin": "2025-06-15",
   "pc_StatementEnd": "2025-06-15",
   "pcla_AdjustmentGroup": "CO",
   "pcla_AdjustmentReason": "29",
   "pcla_AdjustmentAmount": 4500.00,
   "pcl_ProcedureCode": "99214",
   "pcl_ChargedAmount": 4500.00,
   "pcl_PaidAmount": 0.00  }

// 837 Data (Original Claim)
{  "ec_ClaimNo": "CLM-2026-00142",
   "ec_PayerName": "Blue Cross Blue Shield",
   "ec_PayerID": "BCBS-IL",
   "ec_InsuranceType": "Commercial",
   "ec_ServiceDateFrom": "2025-06-15",
   "ec_PrincipalDiagnosis": "J06.9",
   "ec_BillProvNPI": "1234567890",
   "ec_DelayReasonCode": "",
   "ec_ClaimFrequency": "1",
   "ec_SubscriberID": "XYZ123456" }
```

Claim B — Missing Information Denial

```
// 835 Data
{  "pc_ClaimID": "CLM-2026-00287",
   "pc_ClaimStatus": "4",
   "pc_ClaimAmount": 12800.00,
   "pc_ClaimPaid": 0.00,
   "pc_InsuranceType": "Medicare",
   "pc_ReceivedDate": "2026-02-10",
   "pcla_AdjustmentGroup": "CO",
   "pcla_AdjustmentReason": "16",
   "pcla_AdjustmentAmount": 12800.00,
   "pcl_ProcedureCode": "27447",
   "pcl_ProcedureModifier1": "",
   "pcl_RemarkCodes": "N20" }
```

```
// 837 Data
{  "ec_ClaimNo": "CLM-2026-00287",
   "ec_PayerName": "Medicare Part B",
   "ec_InsuranceType": "Medicare",
   "ec_ServiceDateFrom": "2026-01-08",
   "ec_PrincipalDiagnosis": "M17.11",
   "ec_PriorAuthorization": "AUTH-998877",
   "ec_BillProvNPI": "9876543210",
   "ec_TypeOfBill": "131",
   "ec_ClaimFrequency": "1" }
```

Claim C — Medical Necessity Denial

```
// 835 Data
{  "pc_ClaimID": "CLM-2026-00391",
   "pc_ClaimStatus": "4",
   "pc_ClaimAmount": 8200.00,
   "pc_ClaimPaid": 0.00,
   "pc_InsuranceType": "Commercial",
   "pcla_AdjustmentGroup": "CO",
   "pcla_AdjustmentReason": "50",
   "pcla_AdjustmentAmount": 8200.00,
   "pcl_ProcedureCode": "72148",
   "pcl_RemarkCodes": "N386" }

// 837 Data
{  "ec_ClaimNo": "CLM-2026-00391",
   "ec_PayerName": "Aetna",
   "ec_InsuranceType": "Commercial",
   "ec_ServiceDateFrom": "2026-02-20",
   "ec_PrincipalDiagnosis": "M54.5",
   "ec_Diag2": "M51.16",
   "ec_PriorAuthorization": "",
   "ec_BillProvNPI": "5678901234",
   "ec_RendProvSpecialty": "Radiology" }
```

Claim D — Duplicate Claim Denial

```
// 835 Data
{  "pc_ClaimID": "CLM-2026-00455",
   "pc_ClaimStatus": "4",
   "pc_ClaimAmount": 3200.00,
   "pc_ClaimPaid": 0.00,
   "pc_InsuranceType": "Commercial",
   "pcla_AdjustmentGroup": "CO",
   "pcla_AdjustmentReason": "18",
   "pcla_AdjustmentAmount": 3200.00,
   "pcl_ProcedureCode": "99213" }

// 837 Data
{  "ec_ClaimNo": "CLM-2026-00455",
   "ec_PayerName": "United Healthcare",
   "ec_InsuranceType": "Commercial",
```

```
    "ec_ServiceDateFrom": "2026-01-10",
    "ec_ClaimFrequency": "1",
    "ec_PrincipalDiagnosis": "J20.9",
    "ec_BillProvNPI": "1234567890" }
```

5. What to Deliver

5.1 Code Repository

Push to a private GitHub repository and share access with harsh@gabeo.ai. We expect:

- Clean, modular Python code with type hints and docstrings
- A working pipeline that takes claim data in, produces analysis out
- Your prompts stored separately (not buried in code) so we can read and evaluate them
- A synthetic dataset you created for testing (at least 20 claims, mix of paid/denied)
- A README with setup instructions and your design decisions

5.2 README Must Include

- Architecture overview: how your system works end-to-end (diagram is a plus)
- Design decisions: why you chose your approach, what trade-offs you considered
- How you evaluated your system: what metrics you used, what the results look like
- Known limitations and what you would do differently with more time

5.3 Video Walkthrough (Strong Plus)

A short screen recording (under 10 minutes) where you run through your system, show it processing a claim, and explain your key decisions. This is not required, but candidates who include one consistently stand out.

6. How We Evaluate

We care about how you think, not just whether it works. Here is what we weigh:

Dimension	What We Look For	Weight
Problem Understanding	Do you understand the healthcare domain? Can you reason about CARC codes, filing limits, and payer behavior?	25%
System Design	Is your architecture well-reasoned? Can it scale? Is the data flow clean?	25%
AI/ML Quality	Prompt engineering, model selection, output quality, handling of edge cases, hallucination prevention	25%
Engineering Quality	Code structure, error handling, testing approach, documentation, reproducibility	15%
Practical Thinking	Cost awareness, latency considerations, evaluation methodology, production readiness	10%

What Sets Great Submissions Apart

Deep reasoning about denial patterns, not just surface-level CARC code lookup.
A clear evaluation approach: how do you know your system is working well?
Evidence of iteration: showing you improved outputs based on testing.
Thoughtful trade-offs: cost vs. quality, speed vs. accuracy, model selection rationale.
Generating a solid synthetic dataset that covers realistic edge cases.

7. Ground Rules

- You may use any LLM provider (Gemini, OpenAI, Anthropic, open-source). If using paid APIs, keep costs reasonable.
- AI-assisted coding tools are permitted, but you must be able to explain every decision you made.
- Do not share this assignment or your solution publicly.
- When in doubt about requirements, make a reasonable assumption and document it. Good judgment matters.
- Quality over completeness. A well-designed system that solves Problem 1 and 2 deeply is better than a shallow attempt at all four.
- Commit history matters. We want to see how you think, not just a single final commit.

8. Quick Domain Reference

You are not expected to be a healthcare billing expert. Here is enough context to be effective:

EDI 835 (Remittance Advice): Electronic document from the payer explaining how they processed the claim. Think of it as the payer's report card on your bill.

EDI 837 (Claim Submission): The original bill sent by the provider to the payer. Contains everything about the patient visit, diagnoses, and services.

CARC (Claim Adjustment Reason Code): Standard codes explaining WHY a payment was adjusted or denied. There are ~300 codes, but a handful account for most denials.

RARC (Remittance Advice Remark Code): Supplemental codes giving extra context beyond the CARC code.

Timely Filing: Every payer has a deadline for submitting claims (90 days to 1 year). Miss it and the claim is dead. Medicare = 365 days. Commercial varies widely.

NPI (National Provider Identifier): Unique 10-digit ID for every healthcare provider in the US.

CPT/HCPCS Codes: Standardized codes for medical procedures and services (e.g., 99213 = office visit, 27447 = knee replacement).

ICD-10 Codes: Diagnosis codes (e.g., M17.11 = primary osteoarthritis of right knee, J06.9 = acute upper respiratory infection).

Prior Authorization: Some services require payer pre-approval. Denial for lack of prior auth is common and often appealable.

---

9. Submission

1. Push your code to a private GitHub repository.
2. Share repository access with: harsh@gabeo.ai
3. Include video walkthrough link in your README (if applicable).
4. Reply to the assignment email confirming submission with the repo link.

Questions?
Email harsh@gabeo.ai with any questions about the assignment.
Asking good questions is a signal of strength, not weakness.
We are happy to clarify scope, data, or expectations.

Good luck. We are excited to see what you build.
Gabeo AI | gabeo.ai